Kabir Kanha Arora
1710110165
CSD201
Assignment 1

Answer 1

Let us assume that we have created two stacks Stack1 and Stack2, where Stack1 is the stack that is given to us and Stack2 is the stack into which we are inserting the elements after performing **modified insertion sort algorithm.**

We shall be using simple push and pop operations for the same and we assume that those functions have been defined beforehand.

The basic steps to implement the same is as given in the following algorithm:

1. *Initialise Stack1 according to the values taken from the user.*
2. *Initialise Stack2 to NULL.*
3. *Declare two variables tempS1 and tempS2 to hold the recently popped values from Stack1 and Stack2 respectively.*
4. *Repeat the following steps until Stack1 becomes empty, which would mean that all the elements have been sorted and inserted into Stack2*
    a. *Pop out the top value from Stack1 and store it in tempS1.*
    b. *If Stack2 is empty, push the value stored in tempS1 onto Stack2 and go back to step 4(a) after checking for empty condition of Stack1.*
    c. *Pop out the top value from Stack2 and store it in tempS2.*
    d. *If tempS2 > tempS1, push tempS2 back onto Stack2 and then push tempS1 onto Stack2 and go back to step 4(a) after checking for empty condition of Stack1.*
    e. *Otherwise, push the value of tempS2 onto Stack1 and go back to step 4(c).*
5. *Display stack status to the user.*
6. *Finished*

Answer 2

Worst-case time complexity of Bubble Sort algorithm

//Algorithm for BUBBLE SORT

```
for (i=0; i<n; ++i)
        for(j=0; j<n-i-1; ++j)
                if arr[j] > arr[j+1]                    //Runs (n-1)+(n-2)+(n-3)+...+2+1 times
                        Then swap these two elements
```

Ans: We know that sum of first n natural numbers is equal to (n)(n-1)/2

So,

In this case,

Sum = (n-1)(n-2)/2

= (n2 - 3n + 2)/2

= n2/2 -3n/2 +1

The highest degree term is $\mathbf{n^2}$.

We drop the constant coefficient ½ to calculate **O(f(n))**

=> So we get that the worst case time complexity for BUBBLE SORT is $\underline{O(n^2)}$

Answer 3

```
#include <stdio.h>
#include <stdlib.h>

typedef struct node
{
    int val;
    struct node* prev;
    struct node* next;
}node1;
node1* head=NULL;
int count=0;

void add()
{
    count++;
    int n;
    printf("\nEnter element to be added: ");
    scanf("%d",&n);
    if(head==NULL)
    {
        head=(node1*)malloc(sizeof(node1));
        if(head==NULL)
        {
            printf("\nMemory not assigned");
            exit(0);
        }
        else
        {
            head->val = n;
            head->next = NULL;
            head->prev=NULL;
        }
    }
```

```
 else
    {
        node1* temp=(node1*)malloc(sizeof(node1));
        if(temp==NULL)
        {
            printf("\nMemory not assigned");
            exit(0);
        }
        else
        {
            temp->val=n;
            temp->next=NULL;
        }
        node1* ptr=head;
        while(ptr->next!=NULL)
            ptr=ptr->next;
        ptr->next=temp;
        temp->prev=ptr;
    }
}

void display()
{
    node1* ptr = head;
    printf("\n\n");
    while(ptr!=NULL)
    {
        printf("%d ",ptr->val);
        ptr=ptr->next;
    }
}

void sort() {
    if(count==0||count==1)
        exit(0);
    node1 *i = head->next, *j;
    while (i != NULL) {
        int key = i->val;
        j = i->prev;
        while (j != NULL && j->val > key) {
            j->next->val = j->val;
            j = j->prev;
        }
        if(j==NULL) {
            j = head;
            j->val=key;
        }
        else
            j->next->val = key;
        i=i->next;
    }
}
```

```c
void main()
{
    start:
    printf("\nWhat would you like to do?");
    printf("\n1. Add");
    printf("\n2. Sort");
    printf("\n3. Display");
    printf("\n4. Exit");
    int ch;
    enter:
    printf("\n\nEnter your choice: ");
    scanf("%d", &ch);
    switch(ch)
    {
        case 1:{
            add();
            break;
        }
        case 2:{
            sort();
            break;
        }
        case 3:{
            display();
            break;
        }
        case 4:{
            exit(0);
        }
        default:{
            printf("\nInvalid choice!");
            goto enter;
        }
    }
    goto start;
}
```

Time Complexity Calculation

The worst case calculation for O comes from the case when the list is sorted, but in descending order. Let the total list contain **n** elements.
In that case, every element is lesser than every element of the sub list to its left. In general, if we're inserting into a sub list with k elements, all k might have to slide over by one position. If the sub list consists of k elements, then for the first iteration, k=1, then k=2, then k=3 and so on until the last time when k=n-1

So, total cost = c[1+2+3+…(n-2)+(n-1)]

=c.(n-1)(n-2)/2

=c.(n2-3n+2)

So, the time complexity is $O(n^2)$

Answer 4

//Dualstack

```
#include <stdio.h>
#include <stdlib.h>

typedef struct node
{
    int val;
    struct node* prev;
    struct node* next;
}node1;
node1* head=NULL;
int count=0;

void add()
{
    count++;
    int n;
    printf("\nEnter element to be added: ");
    scanf("%d",&n);
    if(head==NULL)
    {
        head=(node1*)malloc(sizeof(node1));
        if(head==NULL)
        {
            printf("\nMemory not assigned");
            exit(0);
        }
        else
        {
            head->val = n;
            head->next = NULL;
            head->prev=NULL;
        }
    }
    else
    {
        node1* temp=(node1*)malloc(sizeof(node1));
        if(temp==NULL)
        {
            printf("\nMemory not assigned");
            exit(0);
        }
        else
        {
            temp->val=n;
            temp->next=NULL;
        }
        node1* ptr=head;
        while(ptr->next!=NULL)
            ptr=ptr->next;
```

```
        ptr->next=temp;
        temp->prev=ptr;
    }
}

void display()
{
    node1* ptr = head;
    printf("\n\n");
    while(ptr!=NULL)
    {
        printf("%d ",ptr->val);
        ptr=ptr->next;
    }
}

void sort() {
    if(count==0||count==1)
        exit(0);
    node1 *i = head->next, *j;
    while (i != NULL) {
        int key = i->val;
        j = i->prev;
        while (j != NULL && j->val > key) {
            j->next->val = j->val;
            j = j->prev;
        }
        if(j==NULL) {
            j = head;
            j->val=key;
        }
        else
            j->next->val = key;
        i=i->next;
    }
}

void main()
{
    start:
    printf("\nWhat would you like to do?");
    printf("\n1. Add");
    printf("\n2. Sort");
    printf("\n3. Display");
    printf("\n4. Exit");
    int ch;
    enter:
    printf("\n\nEnter your choice: ");
    scanf("%d", &ch);
    switch(ch)
    {
        case 1:{
            add();
            break;
        }
        case 2:{
            sort();
            break;
```

```
        }
        case 3:{
            display();
            break;
        }
        case 4:{
            exit(0);
        }
        default:{
            printf("\nInvalid choice!");
            goto enter;
        }
    }
    goto start;
}
```

Why is it not possible for 3 stacks?

The dualstack has a unique implementation. The top of the first stack is at the beginning of the array and the top of the second array is at the end of the array. The dualstack is said to be full when the top of the first stack comes to the element of the array which is just before the element occupied by the top of the second stack. Elements of stack 1 are entered from the beginning of the array towards right and vice versa for the elements of stack 2. This leads to a highly efficient storage system and no memory is wasted.

However,
If we were to try and implement the same scheme of things for 3 stacks, it would lead to a lot of memory wastage. For eg: If we took our triplestack as an array with 30 elements and divided it into three parts of 10 each with the first element of each stack starting from the 1st, 11th and 21st positions, then when one stack fills up, it is likely that the other still have vacant elements but we would be forced to declare the triplestack full as elements of one stack will not be possible to add.
Memory loss would still occur if we start one stack from the beginning, one from the end and one from the middle.

Hence, we can safely say that such a nice implementation is not possible for 3 stacks and if we try to do it, it would be a highly confusing and inefficient method.

Answer 5

//Evaluating an arithmetic expression

```
#include<stdio.h>
#include<stdlib.h>
int numstk[20];
int charstk[20];
int numtop=-1;
int chartop=-1;
```

```
void pushnum(int num)
{
    if(numtop==19)
    {
        printf("\nOVERFLOW");
        exit(0);
    }
    else
        numstk[++numtop]=num;
}

void pushchar(char ch)
{
    if(chartop==19)
    {
        printf("\nOVERFLOW");
        exit(0);
    }
    else
        charstk[++chartop]=ch;
}
void solve(char ch) {
    int m = numstk[numtop-1], n = numstk[numtop], res;
    if (ch == '*')
        res = m * n;
    else if (ch == '/')
        res = m / n;
    else if (ch == '+')
        res = m + n;
    else if (ch == '-')
        res = m - n;
    numtop--;
    numstk[numtop] = res;
}

char pop(char ch) {
    int flag = 0;

    if (ch == '*' || ch == '/' || ch == '+' || ch == '-') {
        while (1) {
            if ((ch == '*' || ch == '/') && (charstk[chartop] == '+' ||
charstk[chartop] == '-'))
                flag = 1;
            if (chartop == -1)
                break;
            else if (charstk[chartop] == '(')
                break;
            else if (flag == 1)
                break;
            solve(charstk[chartop]);
            chartop--;
        }
    } else if (ch == ')') {
        while (charstk[chartop] != '(' && chartop > -1) {
            solve(charstk[chartop]);
            chartop--;
        }
        chartop--;
```

```
    }
}

void main()
{
    char exp[50];
    printf("\nEnter expression: ");
    scanf("%s",exp);
    for(int i=0;exp[i]!='\0';++i) {
        if (exp[i] >= '0' && exp[i] <= '9')
            pushnum(exp[i]);
        else if (exp[i] == '(')
            pushchar(exp[i]);
        else if(exp[i]==' ')
            continue;
        else if(exp[i]=='*'||exp[i]=='/'||exp[i]=='-
'||exp[i]=='+'||exp[i]==')')
            pop(exp[i]);
    }
}
```

Answer 6

//Linked representation of bags

```
#include <stdio.h>
#include <stdlib.h>

typedef struct node
        {
            int val;
            int num;
            struct node* next;
        }node1;
node1* head=NULL;

void add(int n, int cop)
{
    if(head==NULL)
    {
        head=(node1*)malloc(sizeof(node1));
        if(head==NULL)
        {
            printf("\nMemory not assigned");
            exit(0);
        }
        else
        {
            head->val = n;
            head->num=cop;
```

```
            head->next = NULL;
        }
    }
    else
    {
        node1* ptr=head;
        while(ptr!=NULL)
        {
            if(ptr->val==n)
            {
                ptr->num+=cop;
                return;
            }
            else
                ptr=ptr->next;
        }
        node1* temp=(node1*)malloc(sizeof(node1));
        if(temp==NULL)
        {
            printf("\nMemory not assigned");
            exit(0);
        }
        else
        {
            temp->val=n;
            temp->num=cop;
            temp->next=NULL;
        }
        ptr=head;
        while(ptr->next!=NULL)
            ptr=ptr->next;
        ptr->next=temp;
    }
}

void rem(int n, int cop)
{
    node1* ptr=head;
    while(ptr!=NULL)
    {
        if (ptr->val == n)
        {
            if (ptr->num > cop)
            {
                ptr->num -= cop;
                return;
            }
            else
            {
                ptr->num = 0;
                if (ptr == head)
                {
                    head = head->next;
                    free(ptr);
                    return;
                }
            }
        }
```

```
        ptr = ptr->next;
    }
    node1* temp=head;
    while(temp!=NULL&&temp->next!=NULL)
    {
        if(temp->next->num==0)
        {
            node1* del=temp->next;
            temp->next=temp->next->next;
            free(del);
            return;
        }
        temp=temp->next;
    }
}

void subtract(node1* head2)
{
    node1* ptr=head2;
    while(ptr!=NULL)
    {
        rem(ptr->val,ptr->num);
        ptr=ptr->next;
    }
}

void display()
{
    node1* ptr = head;
    printf("\n\n");
    while(ptr!=NULL)
    {
        printf("Element: %d\nCopies: %d\n\n",ptr->val,ptr->num);
        ptr=ptr->next;
    }
}


void main() {
    int ch;
    menu:
    printf("\nWhat would you like to do?");
    printf("\n1. Add");
    printf("\n2. Remove");
    printf("\n3. Subtract");
    printf("\n4. Display");
    printf("\n5. Exit");
    enter:
    printf("\n\nEnter your choice: ");
        scanf("%d",&ch);
    switch (ch) {
        case 1: {
            int num,cop;
            printf("\nWhich element do you wish to add? ");
                scanf("%d",&num);
            printf("\nHow many copies? ");
                scanf("%d",&cop);
```

```
 add(num,cop);
    break;
}
case 2: {
    int num,cop;
    printf("\nWhich element do you wish to remove? ");
        scanf("%d",&num);
    printf("\nHow many copies do you want to remove? ");
        scanf("%d",&cop);
    rem(num,cop);
    break;
}
case 3: {
    node1* head2=NULL;
    int n,cop;
    again:
    printf("\nEnter element to be added to secondary list: ");
        scanf("%d",&n);
    printf("\nEnter number of copies: ");
        scanf("%d",&cop);
    if(head2==NULL)
    {
        head2=(node1*)malloc(sizeof(node1));
        if(head2==NULL)
        {
            printf("\nMemory not assigned");
            exit(0);
        }
        else
        {
            head2->val = n;
            head2->num=cop;
            head2->next = NULL;
        }
    }
    else
    {
        node1* ptr=head2;
        while(ptr!=NULL)
        {
            if(ptr->val==n)
            {
                ptr->num+=cop;
                return;
            }
            else
                ptr=ptr->next;
        }
        node1* temp=(node1*)malloc(sizeof(node1));
        if(temp==NULL)
        {
            printf("\nMemory not assigned");
            exit(0);
        }
```

```
                else
                 {
                     temp->val=n;
                     temp->num=cop;
                     temp->next=NULL;
                 }
                 ptr=head2;
                 while(ptr->next!=NULL)
                     ptr=ptr->next;
                 ptr->next=temp;
            }
            char YorN;
            printf("\nDo you wish to add another element?(Y/N): ");
                scanf(" %c",&YorN);
            if(YorN=='Y'||YorN=='y')
                goto again;
            subtract(head2);
            display();
            break;
        }
        case 4: {
            display();
            break;
        }
        case 5: {
            exit(0);
        }
        default: {
            printf("\nInvalid choice");
            goto enter;
        }

    }
    goto menu;
}
```